

Projet SM604 – Partie 1 : Classification MNIST

Mathématiques pour le Machine Learning, EFREI Paris, 2025-2026

Sabrina El Hassani, Aude Labat, Thomas Duriaud, Paul Fontaine, Evan Ladeira

Parties 1 et 2 réalisées from scratch (NumPy). CNN partie 2 : PyTorch (option B).

1 Préparation des données

Les 70 000 images (28×28 pixels) sont représentées par des vecteurs $\vec{x} \in \mathbb{R}^{784}$ et normalisées en divisant par 255. Sans cette étape, les scores $o_k = \sum_j a_{k,j} x_j + b_k$ atteignent plusieurs milliers, rendant e^{o_k} numériquement infini dans le softmax et faisant exploser les gradients lors de la descente. Le jeu est séparé en 60 000 images d'entraînement et 10 000 de test. Les étiquettes sont encodées en vecteurs one-hot $y_i^{(k)} \in \{0, 1\}$ avec $\sum_k y_i^{(k)} = 1$, nécessaires pour la cross-entropy.

2 Modèle linéaire

Pour chaque image, on calcule $O = XA^T + b$, où $A \in \mathbb{R}^{10 \times 784}$ (la transposée est nécessaire car $X \in \mathbb{R}^{n \times 784}$ et on souhaite $O \in \mathbb{R}^{n \times 10}$: $(n \times 784) \cdot (784 \times 10) = (n \times 10)$), $b \in \mathbb{R}^{10}$, puis $P = \text{softmax}(O)$ et $\hat{y} = \arg \max_k P_k$ (7850 paramètres). La cross-entropy est choisie car, contrairement à la MSE, son gradient combiné au softmax ne fait pas intervenir la dérivée de l'activation, évitant la saturation du gradient lors de la descente.

Calcul du gradient. On développe $\log P_{i,k} = o_{i,k} - \log \sum_r e^{o_{i,r}}$ et on substitue dans la loss $L_i = - \sum_k y_i^{(k)} \log P_{i,k}$:

$$L_i = - \sum_k y_i^{(k)} o_{i,k} + \log \left(\sum_r e^{o_{i,r}} \right) \underbrace{\sum_k y_i^{(k)}}_{=1} = - \sum_k y_i^{(k)} o_{i,k} + \log \left(\sum_r e^{o_{i,r}} \right).$$

La dérivée par rapport à $o_{i,k}$ (règle de la chaîne u'/u sur le terme logarithmique) :

$$\frac{\partial L_i}{\partial o_{i,k}} = -y_i^{(k)} + \frac{e^{o_{i,k}}}{\sum_r e^{o_{i,r}}} = P_{i,k} - y_i^{(k)}.$$

On pose $\delta = P - Y \in \mathbb{R}^{n \times 10}$. Comme $\partial o_{i,k} / \partial a_{k,j} = x_{i,j}$ et $\partial o_{i,k} / \partial b_k = 1$, la règle de la chaîne $\frac{\partial L}{\partial \theta} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial \theta}$ donne :

$$\nabla_A L = \frac{1}{n} (P - Y)^T X \in \mathbb{R}^{10 \times 784}$$

$$\nabla_b L = \frac{1}{n} \sum_{\text{lignes}} (P - Y) \in \mathbb{R}^{10}.$$

Mise à jour : $A \leftarrow A - \eta \nabla_A L$, $b \leftarrow b - \eta \nabla_b L$.

Entraînement. On utilise des mini-batches de 256 images plutôt que le gradient total sur les 60 000 : chaque mini-batch produit une mise à jour de A , soit 234 corrections par epoch au lieu d'une seule, ce qui accélère significativement la convergence. L'arrêt se fait sur $|\Delta L| < 10^{-4}$. Après comparaison de $\eta \in \{0,01; 0,1\}$ (cf. Annexe A), $\eta = 0,1$ est retenu : convergence 4,7× plus rapide pour la même accuracy finale. Le modèle linéaire étant convexe, les deux valeurs convergent vers le même minimum global ; seule la vitesse diffère.

3 Modèles avec couches cachées

Architectures. $H = 1$: couche cachée de $p_1 = 128$ neurones, $A^1 \in \mathbb{R}^{128 \times 784}$, $A^2 \in \mathbb{R}^{10 \times 128}$, 101 770 paramètres. $H = 2$: deux couches $p_1 = 128$, $p_2 = 64$, $A^3 \in \mathbb{R}^{10 \times 64}$, 109 386 paramètres. $p_1 = 128$ est une puissance de 2, ce qui est efficace en termes de calcul matriciel ; c'est aussi suffisamment grand pour capturer des représentations intermédiaires sans alourdir l'entraînement. $p_2 = 64$ réduit la dimension avant la sortie tout en restant dans le même ordre de grandeur. ReLU est choisie comme activation car sa dérivée vaut 1 pour $t > 0$ (contre $\sigma(t)(1 - \sigma(t)) \leq 0,25$ pour sigmoid), ce qui évite que le gradient s'atténue exponentiellement à travers les couches.

Rétropropagation $H = 1$. Passage avant : $o^1 = X(A^1)^T + b^1$, $z^1 = \text{ReLU}(o^1)$, $o^2 = z^1(A^2)^T + b^2$, $P = \text{softmax}(o^2)$. Couche de sortie — règle de la chaîne avec $\partial o_{i,k}^2 / \partial a_{k,q}^2 = z_{i,q}^1$:

$$\delta^2 = P - Y, \quad \nabla_{A^2} L = \frac{1}{n} (\delta^2)^T z^1 \in \mathbb{R}^{10 \times 128}, \quad \nabla_{b^2} L = \frac{1}{n} \sum_{\text{lignes}} \delta^2 \in \mathbb{R}^{10}.$$

z^1 influence L via o^2 , donc $\partial L / \partial z^1 = \delta^2 A^2 \in \mathbb{R}^{n \times 128}$. Comme $z^1 = \text{ReLU}(o^1)$ avec $\text{ReLU}'(t) = \mathbf{1}[t > 0]$, la règle de la chaîne donne :

$$\delta^1 = (\delta^2 A^2) \times \text{ReLU}'(o^1) \in \mathbb{R}^{n \times 784},$$

où $\times$ est le produit terme à terme. Couche cachée avec $\partial o_{i,q}^1 / \partial a_{q,j}^1 = x_{i,j}$:

$$\boxed{\nabla_{A^1} L = \frac{1}{n} (\delta^1)^T X \in \mathbb{R}^{128 \times 784}}, \quad \boxed{\nabla_{b^1} L = \frac{1}{n} \sum_{\text{lignes}} \delta^1 \in \mathbb{R}^{128}}.$$

Rétropropagation $H = 2$. Passage avant : $z^2 = \text{ReLU}(z^1(A^2)^T + b^2)$, $P = \text{softmax}(z^2(A^3)^T + b^3)$. On pose $\delta^3 = P - Y$ et on applique le même schéma :

$$\boxed{\nabla_{A^3} L = \frac{1}{n} (\delta^3)^T z^2}, \quad \delta^2 = (\delta^3 A^3) \times \text{ReLU}'(o^2), \quad \boxed{\nabla_{A^2} L = \frac{1}{n} (\delta^2)^T z^1},$$

$$\delta^1 = (\delta^2 A^2) \times \text{ReLU}'(o^1), \quad \boxed{\nabla_{A^1} L = \frac{1}{n} (\delta^1)^T X}.$$

Les biais se mettent à jour comme pour $H = 1$. Ce schéma correspond à la Proposition 5.2 du cours.

Résultats.

Modèle	Paramètres	Erreur train	Erreur test	Accuracy	Écart
Linéaire	7 850	7,50 %	7,80 %	92,20 %	0,30 %
$H = 1$ ($p_1 = 128$)	101 770	1,07 %	2,39 %	97,61 %	1,32 %
$H = 2$ ($p_1 = 128, p_2 = 64$)	109 386	0,33 %	2,43 %	97,57 %	2,10 %

Table 1: Résultats ($\eta = 0,1$, mini-batches 256, cf. Annexe B).

$H = 1$ gagne 5,41 points sur le test par rapport au modèle linéaire. $H = 2$ présente de l'overfitting (écart 2,10 %) : l'erreur train descend à 0,33 % tandis que l'erreur test stagne à 2,43 %. Nous avons testé $p_2 = 32$ (cf. Annexe G) : l'écart reste à 2,25 %, car A^1 concentre 100 352 des 109 386 paramètres. Nous avons également testé un arrêt anticipé sur l'écart train/test : le résultat est identique à la convergence naturelle. On conclut que l'overfitting de $H = 2$ provient du volume de paramètres de la première couche, non de p_2 .

4 Analyse des erreurs et discussion

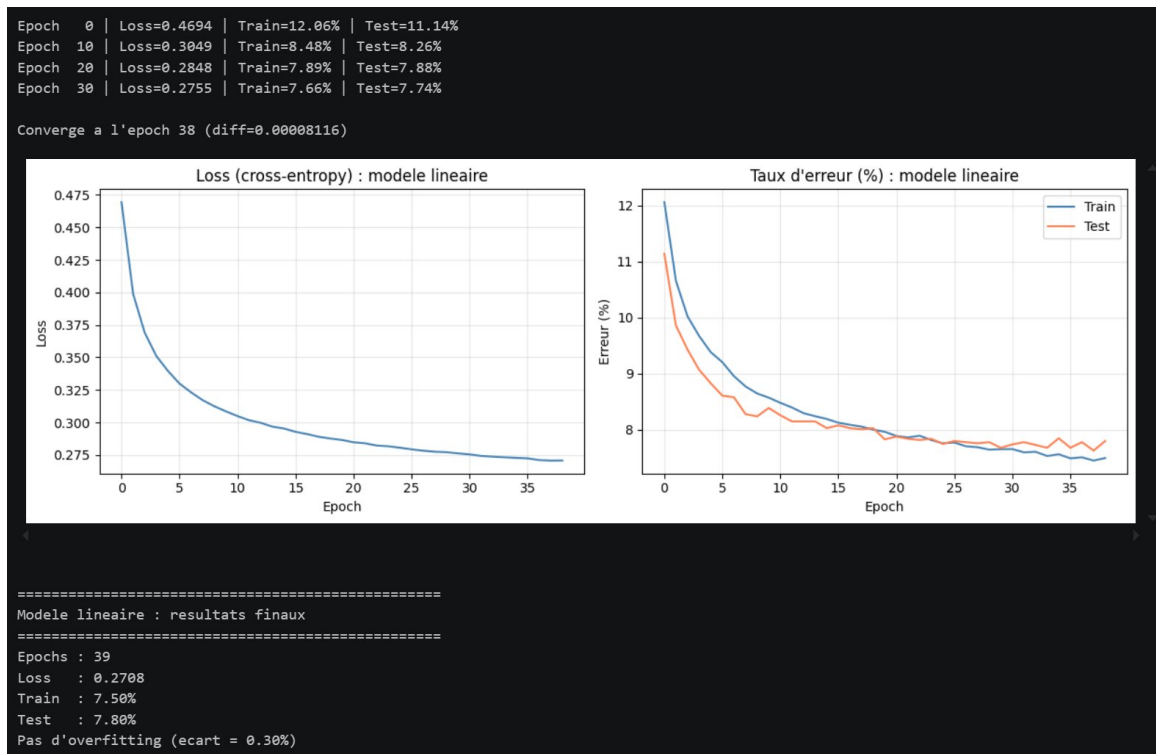
La matrice de confusion (cf. Annexe C) révèle les confusions principales : 5→3 (14 fois), 4→9 (12 fois), 8→3 (10 fois). La projection PCA (cf. Annexe D) confirme que les classes 3, 5, 6 et 8 se superposent au centre du nuage (17,5 % de variance expliquée), tandis que 0 et 1 restent bien séparés. Ces chiffres partagent des formes proches que le modèle ne peut distinguer en traitant les 784 valeurs du vecteur $\vec{x}$ indépendamment, sans information de voisinage spatial. Cela explique également les erreurs à haute confiance (un 2 prédit comme 7 à 92 %, cf. Annexe E).

Transition. MNIST bénéficie de conditions idéales (fond uniforme, chiffres centrés, pas de rotation) permettant d'atteindre 97,6 % sans exploiter la structure spatiale. Sur des images naturelles (CIFAR-10), la relation entre pixels voisins devient indispensable : les filtres convolutifs de la partie 2 analysent des zones 3×3 pour détecter contours et textures, là où ce modèle est aveugle.

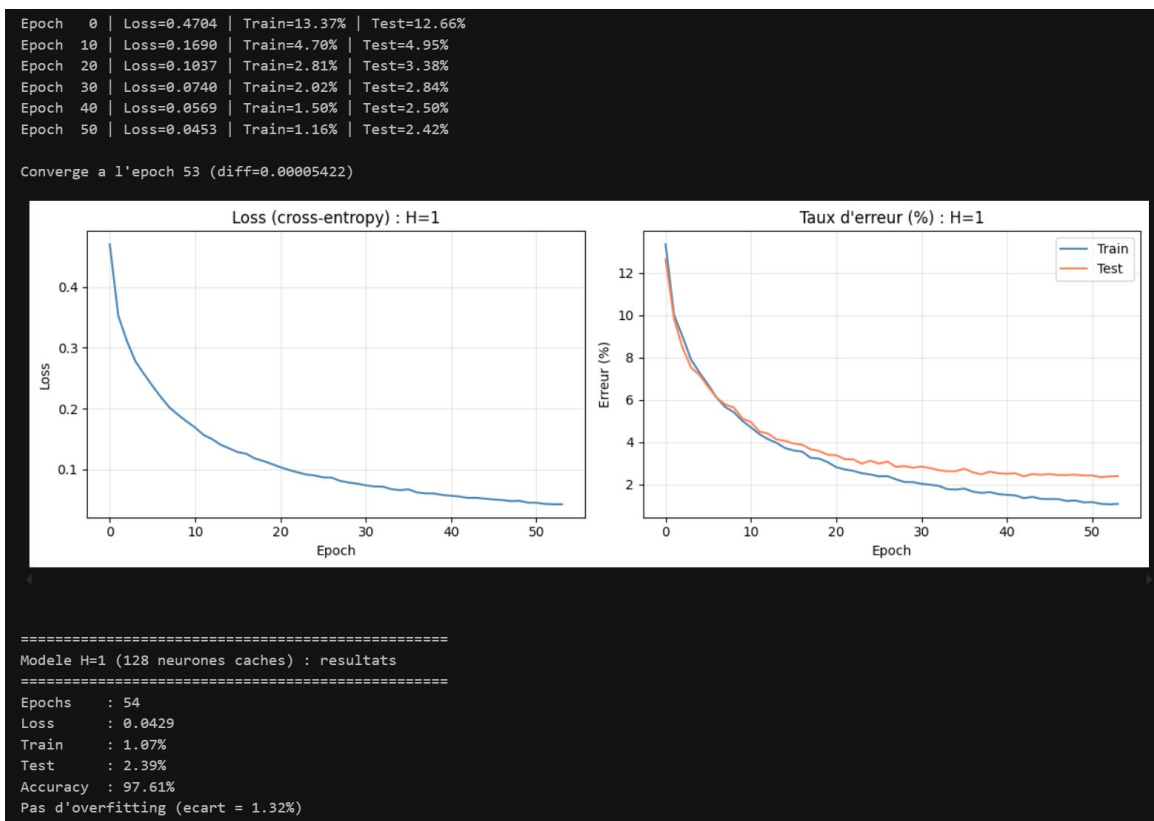
Annexe A : Comparaison des learning rates

η	Epochs	Erreur train	Erreur test	Verdict
0,1	39	7,50 %	7,80 %	Retenu
0,01	184	8,03 %	8,00 %	$4,7\times$ plus lent

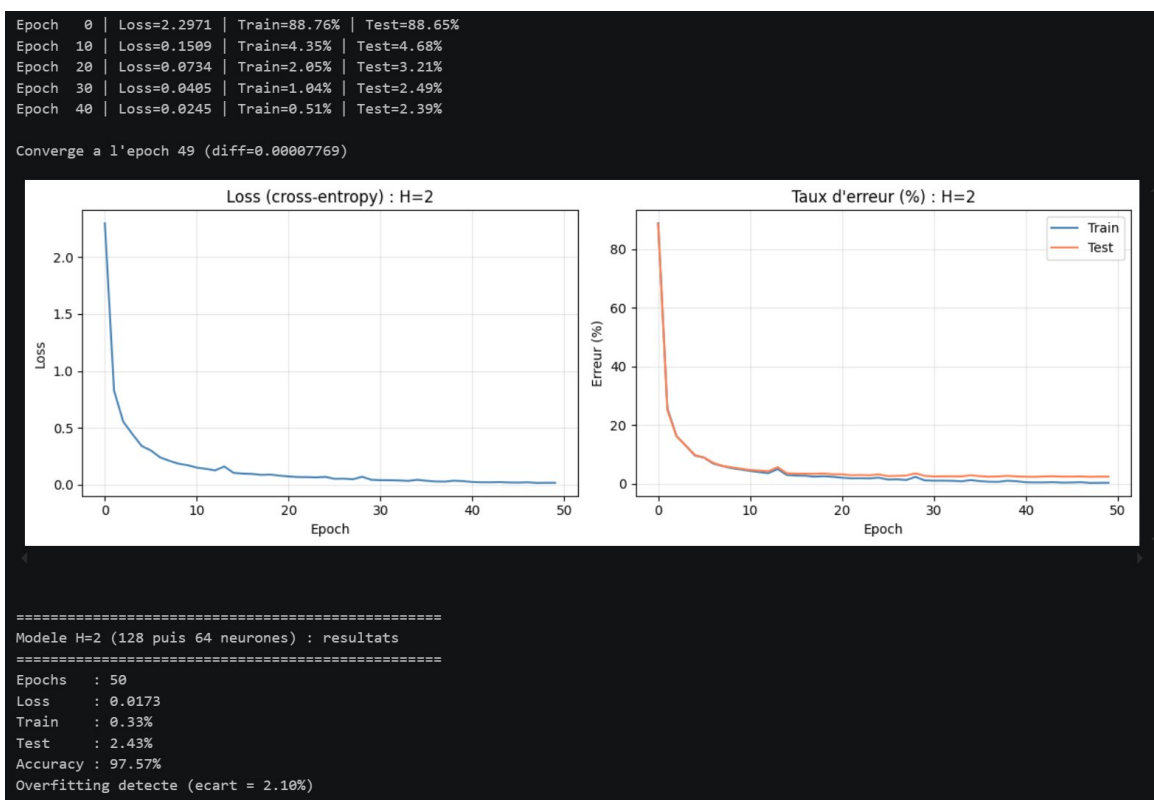
Annexe B : Courbes d'apprentissage



Modèle linéaire : loss et taux d'erreur train/test.

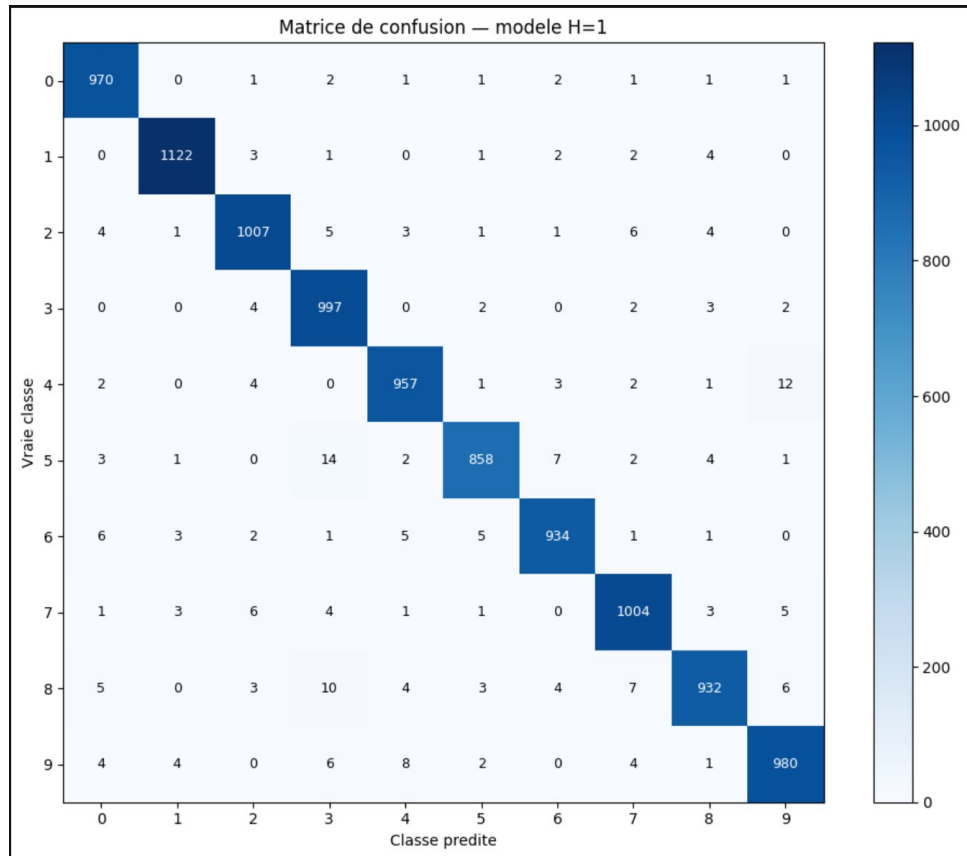


Modèle $H = 1$ ($p_1 = 128$) : loss et taux d'erreur train/test.

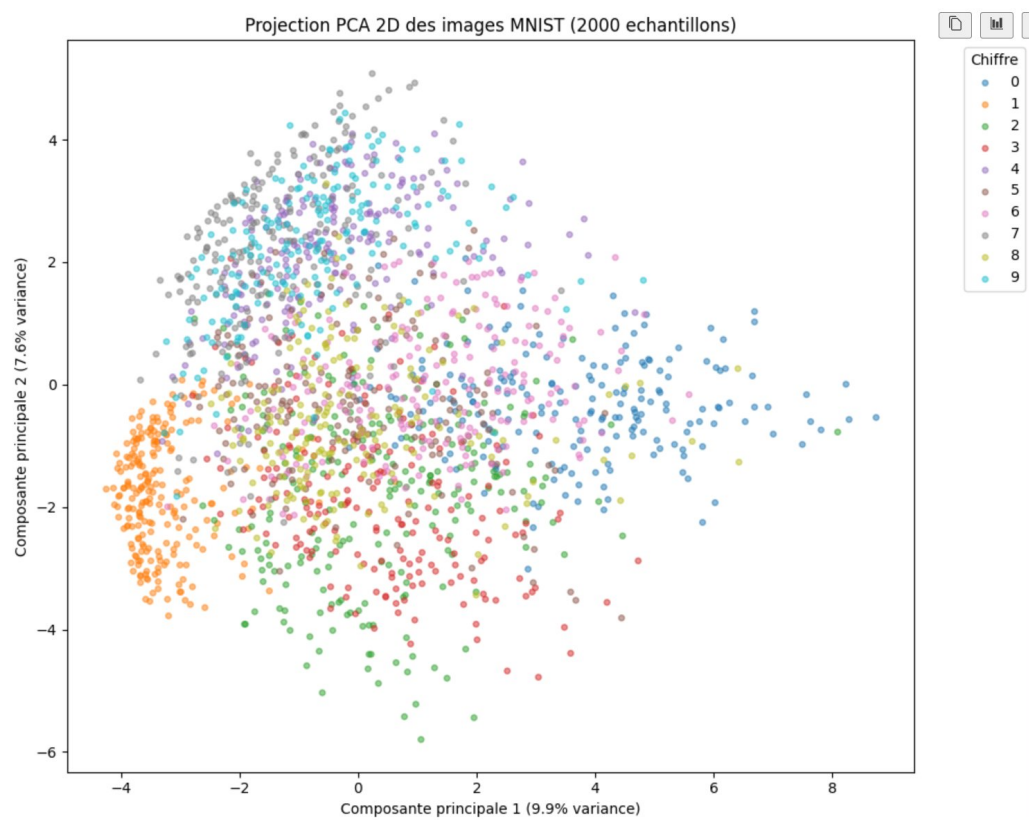


Modèle $H = 2$ ($p_1 = 128$, $p_2 = 64$) : loss et taux d'erreur train/test.

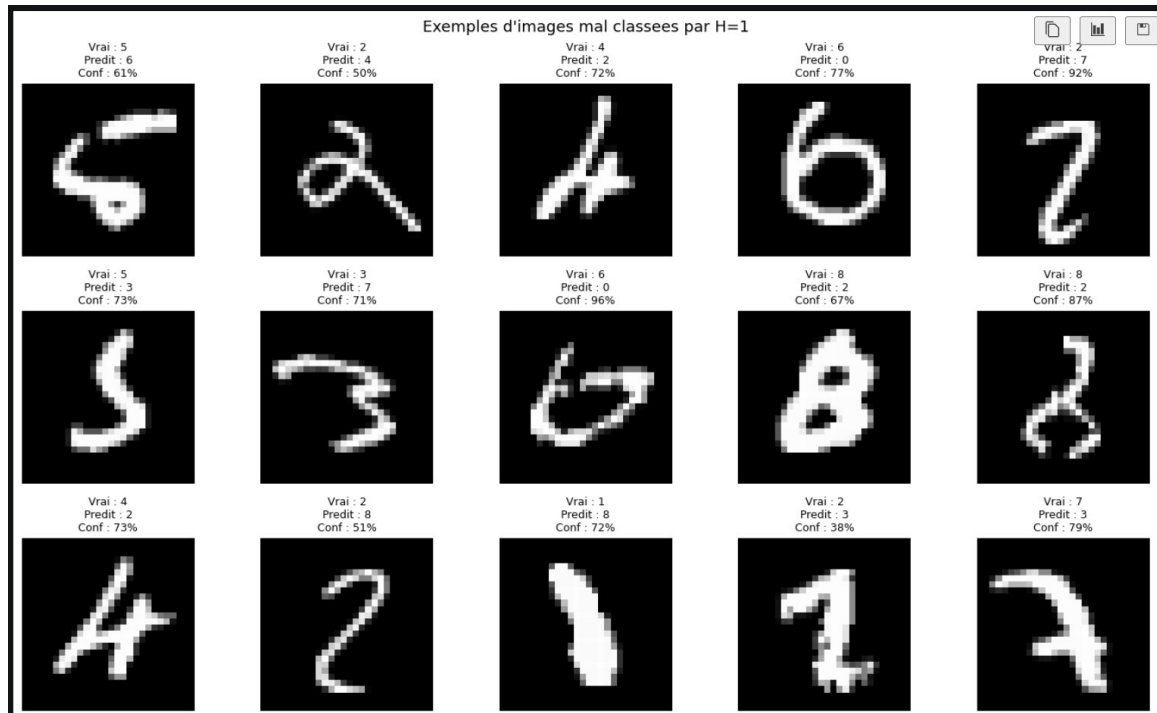
Annexe C : Matrice de confusion (modèle $H = 1$)



Annexe D : Projection PCA 2D (2 000 images de test)



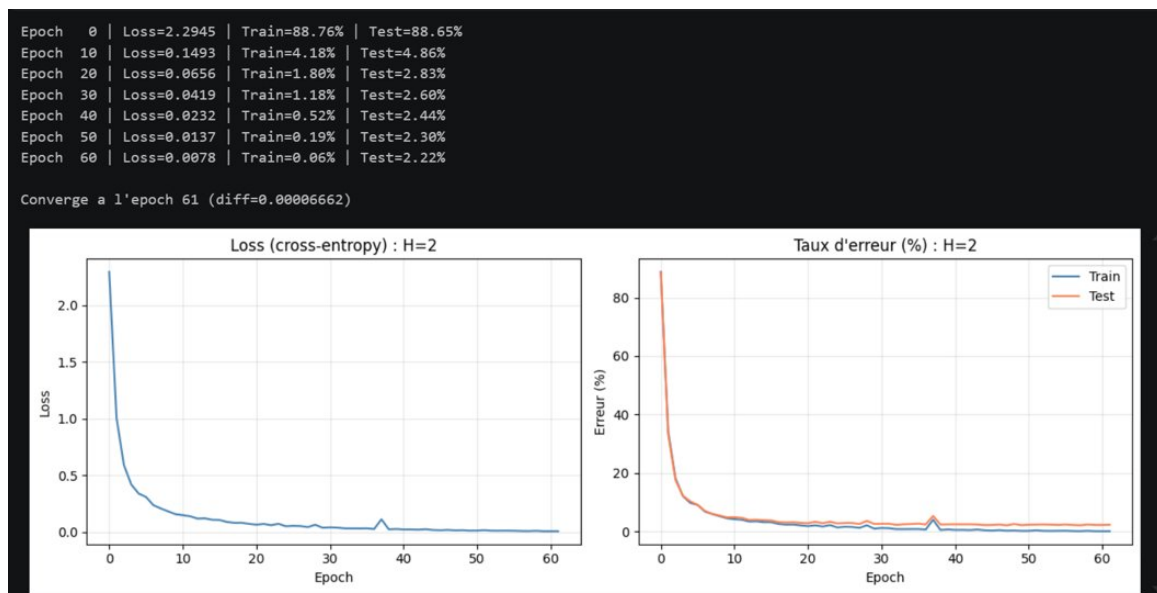
Annexe E : Exemples d'images mal classées (modèle $H = 1$)



Annexe F : Code source

Implémentation from scratch en Python/NumPy disponible dans `MML_Partie1_MNIST.ipynb`.

Annexe G : Test avec $p_2 = 32$ (modèle $H = 2$)



Modèle $H = 2$ avec $p_2 = 32$ (102 282 paramètres) : l'écart train/test reste à 2,25 %, confirmant que l'overfitting provient de A^1 et non de p_2 .